COS 214 Practical 4

TaskForge: Hierarchical Work Processing

Lillian Muller u25253990

Lethabo Molobi u25090209

Francois Venter u25555202

Github: https://github.com/duckyducky2850/Prac4_214

The PDF should contain, in a sensible order:

- system description and team details;
- completed UML class diagram and GoF participant mapping;
- concise design/ownership rationale;
- object diagram and state diagram;
- the three Activity Diagrams;
- the team's traversal-modification policy and any other important design decisions;
- GDB bug investigation and Valgrind evidence;
- GitHub repository link and workflow reflection; and
- a short team contribution statement.

Task 1: Design the System

Before implementation, define the domain and complete the design.

Your submitted design must include:

- a concise description of the chosen domain and the problem your system solves;

Chosen domain:

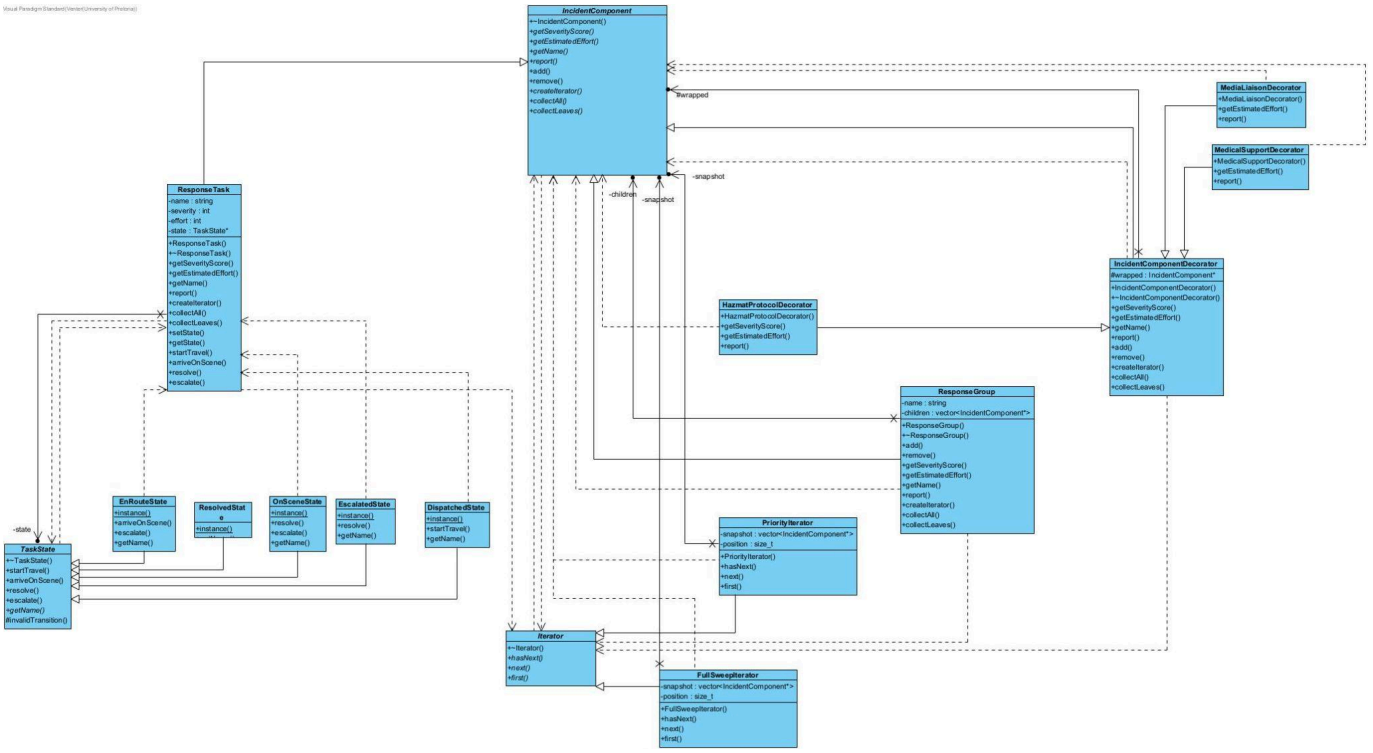
TaskForge models coordination of an emergency response effort. A single Incident is the root of a nested body of work: it contains one or more Divisions (e.g. Fire Division, Medical Division), each Division contains one or more Crews/Squads (e.g. Engine 12 Crew, Triage Unit B), and each Crew contains individual ResponseTasks — the actionable units of work (e.g. “Search Building 3”, “Treat casualty #4”). This gives a genuine recursive part-whole hierarchy at least three levels deep below the root, mixing both nested groups and individual leaf tasks at multiple points.

Problem we solve:

During a live incident, command staff need to:

1. see the whole operation at a glance regardless of how deeply nested it is
2. get a filtered, prioritised list of only the most urgent outstanding work
3. track each task through a real lifecycle so that invalid actions (e.g. marking a task on-scene before it has even started travelling) are caught rather than silently corrupting state
4. attach extra, combinable responsibilities to a task or crew on the fly when circumstances change (a hazmat leak is discovered, a casualty needs both medical support and media handling), and
5. restructure the operation while it is running (reassigning a task to a different crew, escalating it) without breaking a report that is already in progress elsewhere.

- a complete UML class diagram drawn from scratch;



- identification of the GoF participants for Iterator, Composite, State and Decorator in your own design;

Iterator:
-Iterator: Iterator
-Concrete Iterator: FullSweepIterator, PriorityIterator
-Aggregate: IncidentComponent
-Concrete Aggregate: ResponseGroup, ResponseTask

Composite:
-Component: IncidentComponent

-Leaf: ResponseTask
-Composite: ResponseGroup
-Client: main.cpp

State:
-Context: ResponseTask
-State: TaskState
-Concrete State: DispatchedState, EnRouteState, OnSceneState, ResolvedState, EscalatedState

Decorator:
-Component: IncidentComponent
-ConcreteComponent: ResponseTask
-Decorator: IncidentComponentDecorator
-ConcreteDecoratorA: HazmatProtocolDecorator
-ConcreteDecoratorB: MedicalSupportDecorator
-ConcreteDecoratorC: MediaLiaisonDecorator

- relevant inheritance, associations, multiplicities and ownership relationships;
- enough concrete domain classes to make the hierarchy and runtime behaviour non-trivial; and
- a short rationale for the important design and ownership decisions your team made.

Rationale:

Why one ResponseGroup class for both Division and Crew

Divisions and crews behave identically as containers - both aggregate severity as the worst-case among children and effort as the sum of children's effort, and both need add/remove/traversal. Introducing separate ResponseTeam and Squad subclasses with no behavioural difference would add classes without adding design value, which the brief explicitly discourages. If a genuine behavioural difference between the two levels emerges, ResponseGroup is the natural place to split from.

Ownership policy

●A ResponseGroup owns every child added to it and deletes the whole subtree recursively in its destructor.

●ResponseGroup::remove() detaches without deleting. This is what makes moving a task between groups safe: remove from the old parent, add to the new one, and the object is destroyed exactly once, by whichever group still owns it when the tree is torn down.

●An IncidentComponentDecorator owns whatever it wraps and deletes it in its own destructor, so decorating a task doesn't change who is ultimately responsible for freeing it - the decorator chain is simply inserted into the same ownership chain the tree already uses.

●TaskState concrete states are stateless singletons (function-local static), never heap-allocated and never owned by ResponseTask, so there is nothing to leak and no lifetime to manage for objects that live for the whole program and hold no per-task data.

Why traversal hooks (collectAll / collectLeaves) instead of exposing the container

ResponseGroup deliberately has no getChildren(). Client code (and even the iterators themselves) never obtains the raw std::vector used internally - collectAll/collectLeaves are called polymorphically so each node contributes itself (or not, for collectLeaves on a group) without any caller needing to know whether it is looking at a ResponseTask, a ResponseGroup, or a decorated component.

Do not design toward a fixed number of classes. The design should be as small as it can reasonably be while still supporting the required behaviours.

Task 2: Implement the Core Model

Implement the object model described by your UML.

Your implementation must demonstrate all of the following somewhere in the running system:

- a genuine recursive part-whole hierarchy with meaningful leaf and grouping behaviour;
- traversal without exposing the hierarchy's internal representation to client code;
- at least two meaningful traversal behaviours over the same hierarchy;
- two independent traversals over the same runtime structure;
- state-dependent behaviour with valid and invalid transitions;
- at least two meaningful decorators that can be applied at runtime, including a case where decorators are stacked; and
- sensible ownership and clean polymorphic destruction.

Using STL containers internally is allowed. Exposing a container to main.cpp and treating an STL loop as the required Iterator solution is not.

Task 3: Dynamic Behaviour and Design Decisions

Your system should behave like an application rather than a static pattern demonstration.

Create at least two non-trivial runtime scenarios in which several parts of the design collaborate. Across these scenarios, your program must include:

- traversal of nested objects;
- lifecycle/state-dependent behaviour;
- at least one decorated object participating in normal system behaviour; and
- at least one runtime change to the structure, state, decoration, or a combination of these.

Your team must decide and document **how an existing traversal behaves when the structure it refers to changes.** Snapshot traversal, live traversal, invalidation, or another policy may be used, but the policy must be deliberate, safe and consistent with the implementation.

Policy: Snapshot traversal.
Both FullSweepIterator and PriorityIterator capture the structure at the moment they are constructed. FullSweepIterator's constructor calls root->collectAll(snapshot), and PriorityIterator's constructor calls root->collectLeaves(leaves) then filters/sorts into snapshot. From that point on, next()/hasNext() only ever read from that internal std::vector<IncidentComponent*> snapshot - never from the live tree. So an iterator created before a structural change keeps walking the structure exactly as it looked at construction time; it will not see anything added, removed, moved, or decorated afterwards. To observe a change, a new iterator must be created after the change.

Why we chose this policy: it's the safest and easiest option to reason about and defend. A report or triage sweep in progress can't be corrupted or produce inconsistent results just because another part of the system reassigns or modifies a task mid-sweep - which matters in our domain, since dispatch changes can happen at any time during an active incident.

Other design decisions

- Decorators are applied by detaching the plain component from its parent, wrapping it, and re-adding the wrapped result - the parent never needs to know or care whether what it holds is decorated.
- Invalid state transitions print a clear message rather than throwing, so they are visible during the demo without crashing the program; add() / remove() on a leaf or decorator instead throw, since attempting them is a programming error rather than an expected runtime path

---

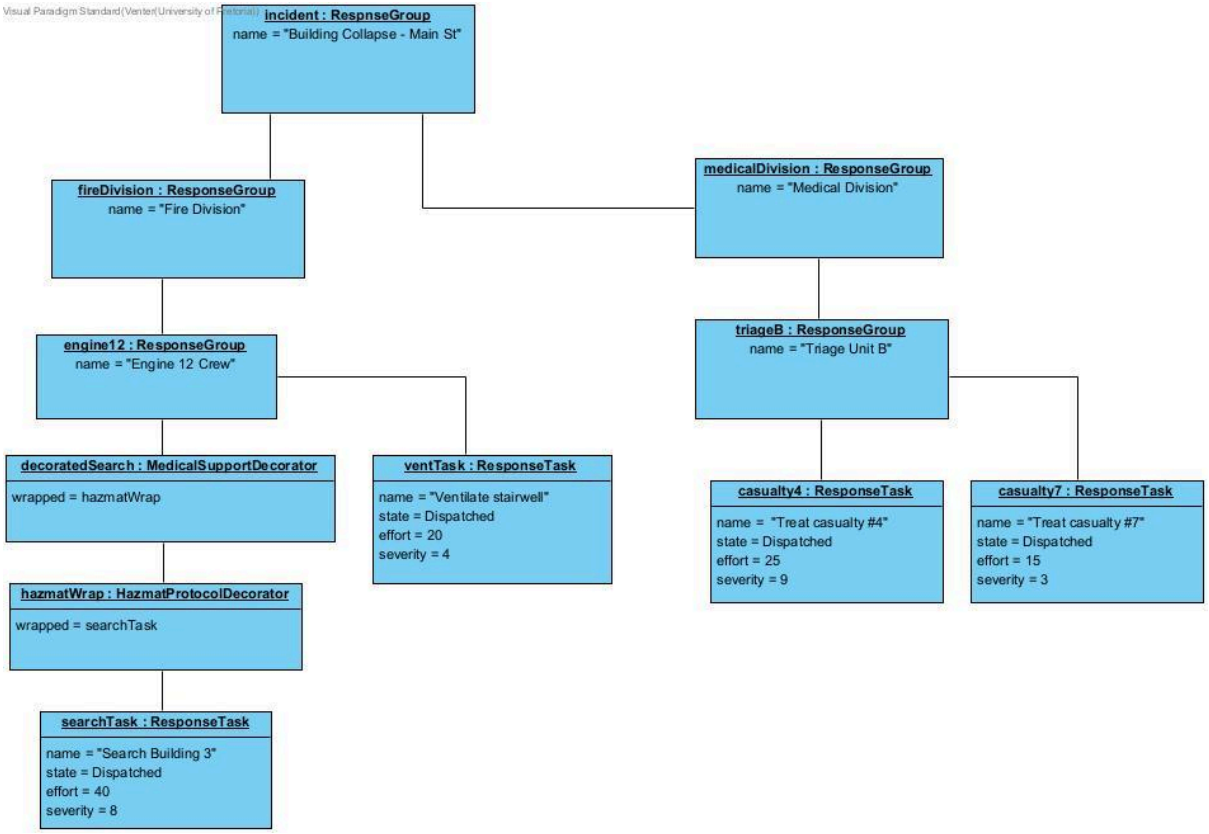
Task 4: UML Diagram Portfolio

Submit a diagram portfolio that explains the design and the important workflows in your actual system.

In addition to the class diagram from Task 1, include:

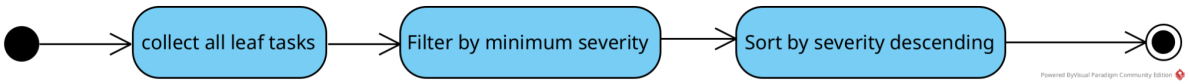
- one UML object diagram showing a meaningful runtime portion of your nested hierarchy;

Object diagram:



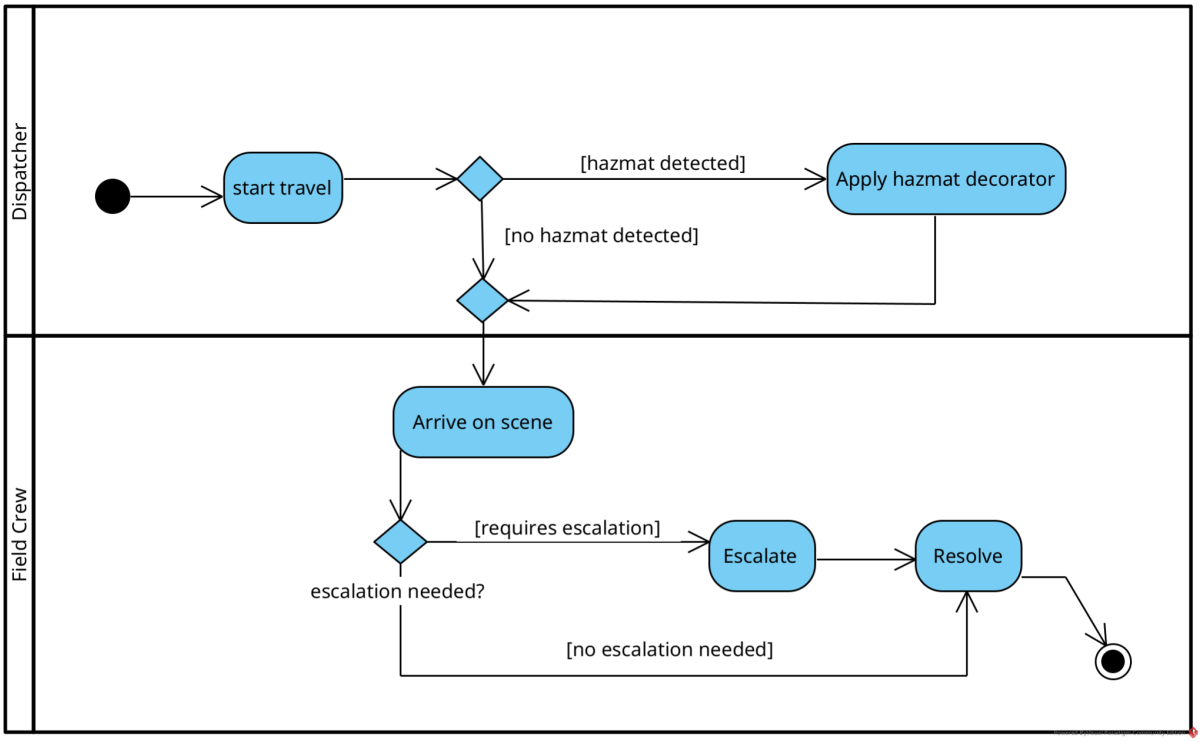
- one UML state diagram showing the lifecycle of an important stateful object; and

State:

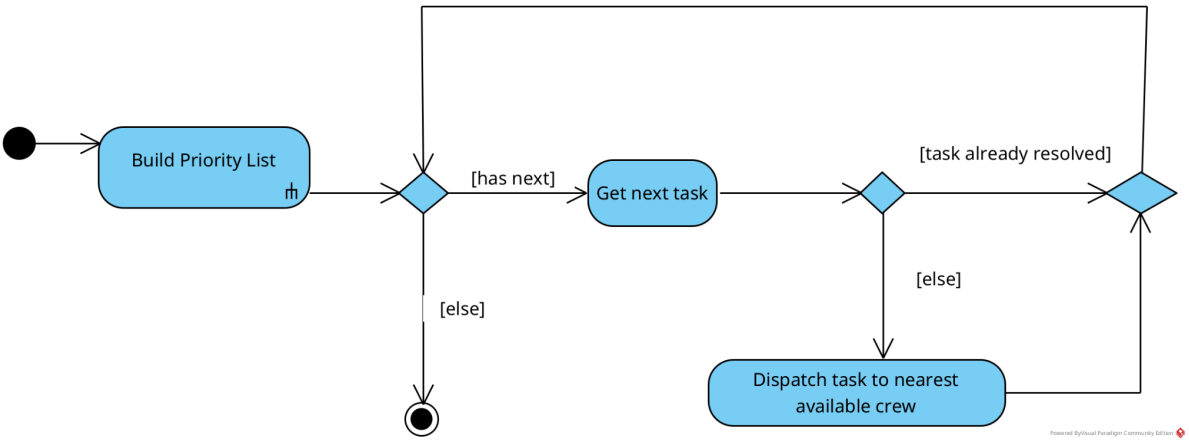


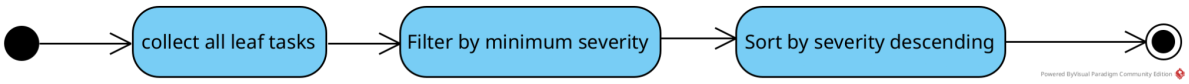
- three substantial UML Activity Diagrams chosen to explain the most important workflows in your system.

Activity diagram 1:

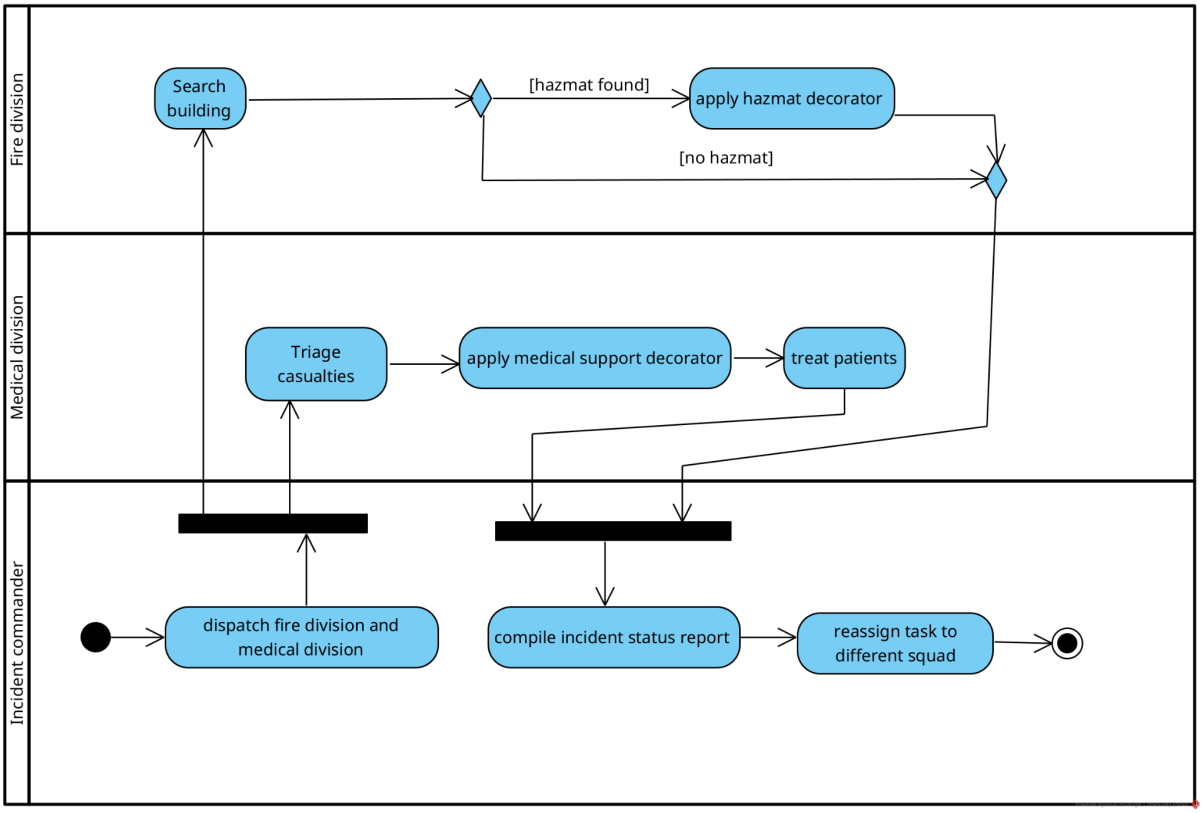


Activity diagram 2:





Activity diagram 3:



The Activity Diagrams should not all tell the same story. Across the three diagrams, demonstrate appropriate use of actions, decisions, guards, merges and loops, as well as at least one fork/join, one called/composite activity, and one use of swimlanes.

At least one Activity Diagram must make traversal visible as part of a larger workflow rather than hiding the entire traversal behind a single "process all" action. At least one should contain meaningful conditional behaviour related to the lifecycle or runtime configuration of an object. The third is your team's choice and should showcase a distinctive workflow from your domain.

The diagrams are part of the design. Important actions and decisions shown in the diagrams must be traceable to the final C++ design and behaviour.

Task 5: Debugging and Memory Investigation

Use the debugging tools introduced in the module as part of development, not only after the program is finished.

Your PDF must contain a short engineering investigation that includes:

- evidence of using GDB on a meaningful part of your program, including breakpoints/stepping and inspection of relevant program state;
- one genuine bug encountered during development, with its symptom, cause, debugging evidence and correction; and

- Valgrind evidence for the final application.

The final implementation must not contain definitely-lost memory originating from your own code.

A typical Valgrind command is:

valgrind --leak-check=full --show-leak-kinds=all ./taskforge

Debugging and Memory Investigation

GDB Session:

A genuine bug encountered during development:

Symptom

main() ran to completion and printed all expected output, then crashed at the very end with Segmentation fault (core dumped) during final cleanup (delete incident;).

Cause

In the reassignment of casualty #7 from Triage Unit B to Engine 12 Crew, the call to triageB->remove(casualty7) was omitted before engine12->add(casualty7)

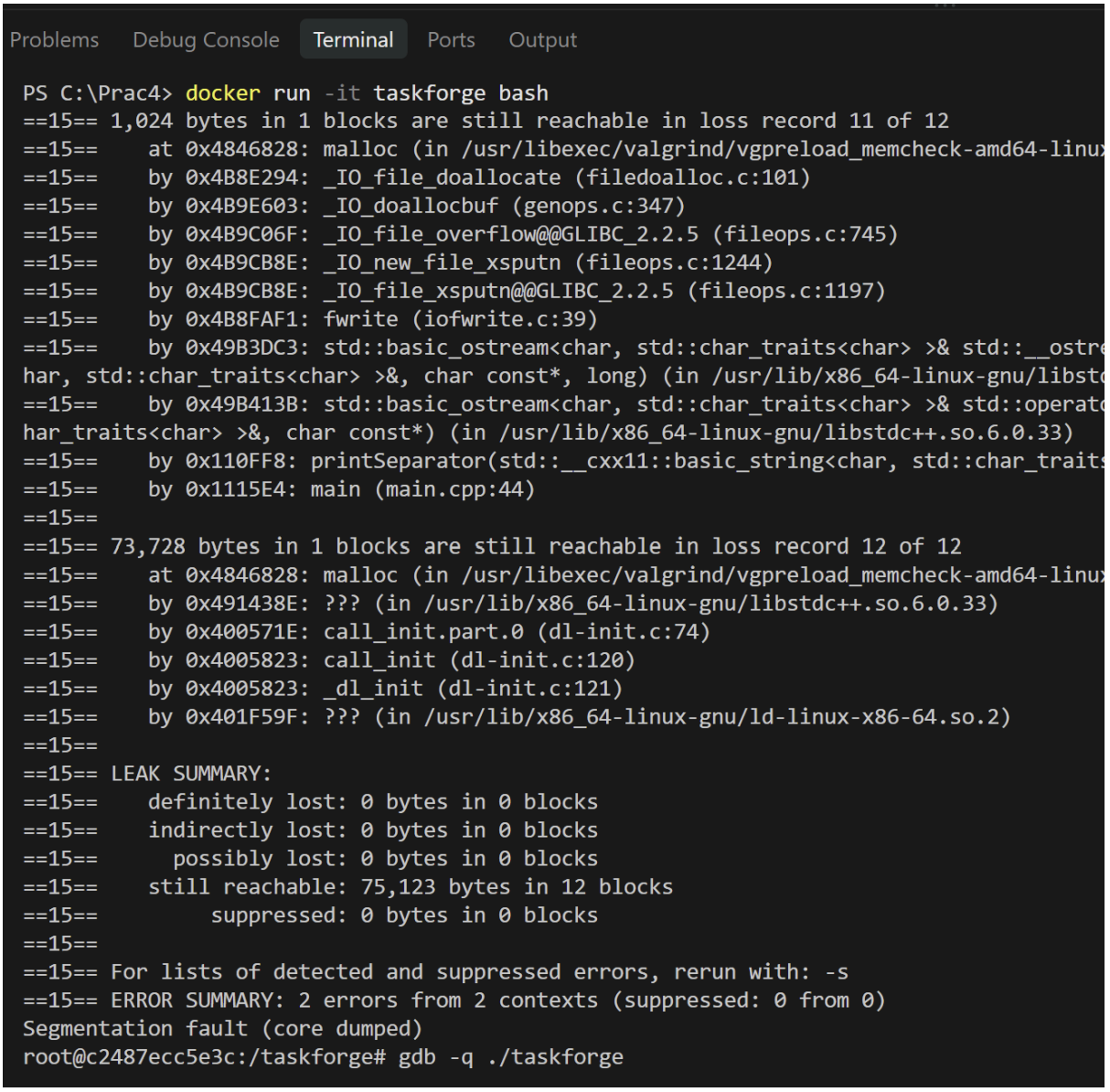
This left the same ResponseTask* stored in two ResponseGroups at once. Both groups' destructors run during the recursive teardown of the tree, so the first group to be destroyed frees casualty7, and the second group's destructor then calls delete on an already-freed pointer — a double free.

Debugging evidence

Valgrind pinpointed an invalid read at the delete child; line inside ResponseGroup's destructor, identified the freed block as having been released by an earlier ResponseTask destructor call from the same line (i.e. the first group's teardown), and traced the block's original allocation back to the exact new ResponseTask(...) call for casualty #7 in main.cpp. The LEAK SUMMARY still reported definitely lost: 0 bytes, confirming this was a double-free / invalid access rather than a leak — a distinct and, in some ways, more serious category of memory bug. GDB's backtrace on the automatically caught SIGSEGV showed the recursive ~ResponseGroup() chain unwinding through Incident → Fire Division → Engine 12 Crew, crashing the second time that chain reached casualty #7 — i.e. exactly the “two groups both think they own this task” mechanism.

```
PS C:\Prac4> docker run -it taskforge bash
==15==
==15== HEAP SUMMARY:
==15==     in use at exit: 75,123 bytes in 12 blocks
==15==   total heap usage: 77 allocs, 65 frees, 77,263 bytes allocated
==15==
==15== 8 bytes in 1 blocks are still reachable in loss record 1 of 12
==15==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreloa
==15==    by 0x10CEF7: std::__new_allocator<IncidentComponent*>::allocate(unsigned lon
==15==    by 0x10CC44: allocate (alloc_traits.h:482)
==15==    by 0x10CC44: std::_Vector_base<IncidentComponent*, std::allocator<IncidentCo
==15==    by 0x10EB19: void std::vector<IncidentComponent*, std::allocator<IncidentCon
_cxx::__normal_iterator<IncidentComponent**, std::vector<IncidentComponent*, std::allo
tor.tcc:459)
==15==    by 0x10EA49: std::vector<IncidentComponent*, std::allocator<IncidentComponen
)
==15==    by 0x10F6EC: ResponseGroup::add(IncidentComponent*) (ResponseGroup.cpp:20)
==15==    by 0x11155D: main (main.cpp:35)
==15==
==15== 16 bytes in 1 blocks are still reachable in loss record 2 of 12
==15==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreloa
==15==    by 0x10CEF7: std::__new_allocator<IncidentComponent*>::allocate(unsigned lon
==15==    by 0x10CC44: allocate (alloc_traits.h:482)
==15==    by 0x10CC44: std::_Vector_base<IncidentComponent*, std::allocator<IncidentCo
==15==    by 0x10EB19: void std::vector<IncidentComponent*, std::allocator<IncidentCon
_cxx::__normal_iterator<IncidentComponent**, std::vector<IncidentComponent*, std::allo
tor.tcc:459)
==15==    by 0x10EA49: std::vector<IncidentComponent*, std::allocator<IncidentComponen
)
==15==    by 0x10F6EC: ResponseGroup::add(IncidentComponent*) (ResponseGroup.cpp:20)
==15==    by 0x111536: main (main.cpp:34)
==15==
==15== 17 bytes in 1 blocks are still reachable in loss record 3 of 12
==15==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreloa
==15==    by 0x49C895A: void std::__cxx11::basic_string<char, std::char_traits<char>,
 char const*, std::forward_iterator_tag) (in /usr/lib/x86_64-linux-gnu/libstdc++.so.6
==15==    by 0x11134C: main (main.cpp:29)
==15==
==15== 22 bytes in 1 blocks are still reachable in loss record 4 of 12
==15==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreloa
==15==    by 0x49C895A: void std::__cxx11::basic_string<char, std::char_traits<char>,
 char const*, std::forward_iterator_tag) (in /usr/lib/x86_64-linux-gnu/libstdc++.so.6
```

```
PS C:\Prac4> docker run -it taskforge bash
==15==    by 0x11134C: main (main.cpp:29)
==15==
==15== 22 bytes in 1 blocks are still reachable in loss record 4 of 12
==15==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreloac
==15==    by 0x49C895A: void std::__cxx11::basic_string<char, std::char_traits<char>, s
 char const*, std::forward_iterator_tag) (in /usr/lib/x86_64-linux-gnu/libstdc++.so.6.0
==15==    by 0x111DCE: main (main.cpp:105)
==15==
==15== 28 bytes in 1 blocks are still reachable in loss record 5 of 12
==15==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreloac
==15==    by 0x49C895A: void std::__cxx11::basic_string<char, std::char_traits<char>, s
 char const*, std::forward_iterator_tag) (in /usr/lib/x86_64-linux-gnu/libstdc++.so.6.0
==15==    by 0x111089: main (main.cpp:19)
==15==
==15== 32 bytes in 1 blocks are still reachable in loss record 6 of 12
==15==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreloac
==15==    by 0x10CEF7: std::__new_allocator<IncidentComponent*>::allocate(unsigned long
==15==    by 0x10CC44: allocate (alloc_traits.h:482)
==15==    by 0x10CC44: std::_Vector_base<IncidentComponent*, std::allocator<IncidentCon
==15==    by 0x10EB19: void std::vector<IncidentComponent*, std::allocator<IncidentComp
_cxx::__normal_iterator<IncidentComponent**, std::vector<IncidentComponent*, std::alloc
tor.tcc:459)
==15==    by 0x10EA49: std::vector<IncidentComponent*, std::allocator<IncidentComponent
)
==15==    by 0x10F6EC: ResponseGroup::add(IncidentComponent*) (ResponseGroup.cpp:20)
==15==    by 0x111FCF: main (main.cpp:115)
==15==
==15== 56 bytes in 1 blocks are still reachable in loss record 7 of 12
==15==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreloac
==15==    by 0x111D9B: main (main.cpp:105)
==15==
==15== 64 bytes in 1 blocks are still reachable in loss record 8 of 12
==15==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreloac
==15==    by 0x111053: main (main.cpp:19)
```


Problems
Debug Console
Terminal
Ports
Output
PS C:\Prac4> docker run -it taskforge bash
==15== 1,024 bytes in 1 blocks are still reachable in loss record 11 of 12
==15==   at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux
==15==   by 0x4B8E294: _IO_file_doallocate (filedoalloc.c:101)
==15==   by 0x4B9E603: _IO_doallocbuf (genops.c:347)
==15==   by 0x4B9C06F: _IO_file_overflow@@GLIBC_2.2.5 (fileops.c:745)
==15==   by 0x4B9CB8E: _IO_new_file_xsputn (fileops.c:1244)
==15==   by 0x4B9CB8E: _IO_file_xsputn@@GLIBC_2.2.5 (fileops.c:1197)
==15==   by 0x4B8FAF1: fwrite (iofwrite.c:39)
==15==   by 0x49B3DC3: std::basic_ostream<char, std::char_traits<char> >& std::__ostre
har, std::char_traits<char> >&, char const*, long) (in /usr/lib/x86_64-linux-gnu/libstd
==15==   by 0x49B413B: std::basic_ostream<char, std::char_traits<char> >& std::operato
har_traits<char> >&, char const*) (in /usr/lib/x86_64-linux-gnu/libstdc++.so.6.0.33)
==15==   by 0x110FF8: printSeparator(std::__cxx11::basic_string<char, std::char_traits
==15==   by 0x1115E4: main (main.cpp:44)
==15==
==15== 73,728 bytes in 1 blocks are still reachable in loss record 12 of 12
==15==   at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux
==15==   by 0x491438E: ??? (in /usr/lib/x86_64-linux-gnu/libstdc++.so.6.0.33)
==15==   by 0x400571E: call_init.part.0 (dl-init.c:74)
==15==   by 0x4005823: call_init (dl-init.c:120)
==15==   by 0x4005823: _dl_init (dl-init.c:121)
==15==   by 0x401F59F: ??? (in /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2)
==15==
==15== LEAK SUMMARY:
==15==   definitely lost: 0 bytes in 0 blocks
==15==   indirectly lost: 0 bytes in 0 blocks
==15==   possibly lost: 0 bytes in 0 blocks
==15==   still reachable: 75,123 bytes in 12 blocks
==15==   suppressed: 0 bytes in 0 blocks
==15==
==15== For lists of detected and suppressed errors, rerun with: -s
==15== ERROR SUMMARY: 2 errors from 2 contexts (suppressed: 0 from 0)
Segmentation fault (core dumped)
root@c2487ecc5e3c:/taskforge# gdb -q ./taskforge

```
PS C:\Prac4> docker run -it taskforge bash
Final state: Resolved
==15== Invalid read of size 8
==15==    at 0x10F623: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:15)
==15==    by 0x10F6B3: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:17)
==15==    by 0x10F631: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:15)
==15==    by 0x10F6B3: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:17)
==15==    by 0x10F631: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:15)
==15==    by 0x10F6B3: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:17)
==15==    by 0x112011: main (main.cpp:128)
==15==  Address 0x4e1b750 is 0 bytes inside a block of size 56 free'd
==15==    at 0x484A164: operator delete(void*) (in /usr/libexec/valgrind/vgpreload_mem
==15==    by 0x110AE7: ResponseTask::~ResponseTask() (ResponseTask.h:20)
==15==    by 0x10F631: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:15)
==15==    by 0x10F6B3: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:17)
==15==    by 0x10F631: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:15)
==15==    by 0x10F6B3: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:17)
==15==    by 0x10F631: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:15)
==15==    by 0x10F6B3: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:17)
==15==    by 0x112011: main (main.cpp:128)
==15==  Block was alloc'd at
==15==    at 0x4846FA3: operator new(unsigned long) (in /usr/libexec/valgrind/vgpreloa
==15==    by 0x111476: main (main.cpp:32)
==15==
==15== Jump to the invalid address stated on the next line
==15==    at 0x0: ???
==15==    by 0x10F6B3: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:17)
==15==    by 0x10F631: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:15)
==15==    by 0x10F6B3: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:17)
==15==    by 0x10F631: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:15)
==15==    by 0x10F6B3: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:17)
==15==    by 0x112011: main (main.cpp:128)
==15==  Address 0x0 is not stack'd, malloc'd or (recently) free'd
==15==
==15==
==15== Process terminating with default action of signal 11 (SIGSEGV)
==15==  Bad permissions for mapped region at address 0x0
==15==    at 0x0: ???
==15==    by 0x10F6B3: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:17)
==15==    by 0x10F631: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:15)
==15==    by 0x10F6B3: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:17)
==15==    by 0x10F631: ResponseGroup::~ResponseGroup() (ResponseGroup.cpp:15)
```

Reading symbols from ./taskforge...

(gdb) run

Starting program: /taskforge/taskforge

```
Program received signal SIGSEGV, Segmentation fault.
0x000055555555b62a in ResponseGroup::~ResponseGroup (
    this=0x555555577540, __in_chrg=<optimized out>)
    at ResponseGroup.cpp:15
15              delete child;
(gdb) bt
#0  0x000055555555b62a in ResponseGroup::~ResponseGroup (
    this=0x555555577540, __in_chrg=<optimized out>)
    at ResponseGroup.cpp:15
#1  0x000055555555b6b4 in ResponseGroup::~ResponseGroup (
    this=0x555555577540, __in_chrg=<optimized out>)
    at ResponseGroup.cpp:17
#2  0x000055555555b632 in ResponseGroup::~ResponseGroup (
    this=0x5555555774d0, __in_chrg=<optimized out>)
    at ResponseGroup.cpp:15
#3  0x000055555555b6b4 in ResponseGroup::~ResponseGroup (
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
```

Correction

The missing detach was added back before the re-add, restoring the single-owner invariant:

triageB->remove(casualty7); // detach before transferring ownership

engine12->add(casualty7);

T his resolves the symptom because casualty7 is now only ever present in one group's child list at a time, so exactly one destructor call frees it, however deep the recursive teardown goes.

```
PS C:\Prac4> docker run -it taskforge bash
  - Fire Division
  - Engine 12 Crew
  - Ventilate stairwell
  - Search Building 3
  - Medical Division
  - Triage Unit B
  - Treat casualty #4
  - Treat casualty #7

===== ITERATOR CREATED AFTER THE MOVE (reflects new structure) =====
  - Building Collapse - Main St
  - Fire Division
  - Engine 12 Crew
  - Ventilate stairwell
  - Search Building 3
  - Treat casualty #4
  - Medical Division
  - Triage Unit B
  - Treat casualty #7

===== PRIORITY TRIAGE (severity >= 5, most urgent first) =====
  - Search Building 3 (severity 10)
  - Treat casualty #4 (severity 9)

===== STATE TRANSITIONS ON A FRESH TASK =====
Initial state: Dispatched
[INVALID TRANSITION] "Assess trapped worker" cannot arriveOnScene() while in state Disp
State is now: OnScene
[INVALID TRANSITION] "Assess trapped worker" cannot startTravel() while in state OnScer
Final state: Resolved
==14==
==14== HEAP SUMMARY:
==14==     in use at exit: 0 bytes in 0 blocks
==14==   total heap usage: 77 allocs, 77 frees, 77,263 bytes allocated
==14==
==14== All heap blocks were freed -- no leaks are possible
==14==
==14== For lists of detected and suppressed errors, rerun with: -s
==14== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
root@96a61f3e4ee5:/taskforge# ^C
root@96a61f3e4ee5:/taskforge# exit
```

Valgrind evidence for the final application:

```
==14==
==14== HEAP SUMMARY:
==14==     in use at exit: 0 bytes in 0 blocks
==14==   total heap usage: 77 allocs, 77 frees, 77,263 bytes allocated
==14==
==14== All heap blocks were freed -- no leaks are possible
==14==
==14== For lists of detected and suppressed errors, rerun with: -s
==14== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Task 6: Docker and GitHub Workflow

The repository is both the development environment and evidence of the team's engineering process.

Your repository must contain at least:

- all source/header files and main.cpp;
- a working Makefile;
- a working Dockerfile;
- README.md; and
- a docs/ directory containing the submitted design material or exported diagrams.

The Docker environment must provide the tools needed to compile, run and investigate the system, including g++, make, gdb and valgrind. The README.md must contain sufficient commands for a marker to reproduce the build and execute the program, GDB and Valgrind without installing project-specific dependencies directly on the host machine.

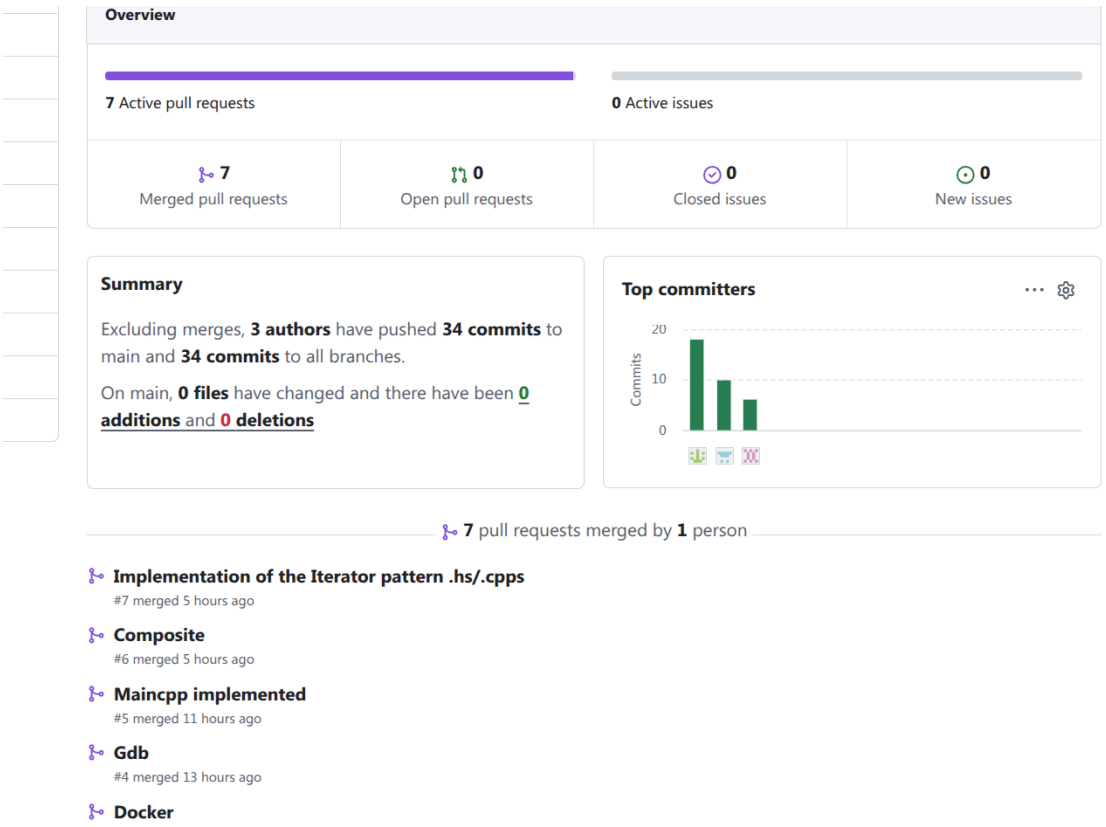
Git history must show genuine work by all three members across the development period. Use meaningful commit messages. Branches, pull requests, issues and reviews are encouraged where useful, but the important requirement is a credible collaborative history rather than a particular Git workflow.

Include a short reflection in the PDF explaining how work was divided and integrated, with references to a few commits, pull requests, issues or other repository activities that demonstrate collaboration.

https://github.com/duckyducky2850/Prac4_214

How work was divided: Code wise, Lethabo did Composite and iterator, Lilly did State and Decorator, and Francoid implemented main. We each created a feature branch for the relevant pattern and then merged it into main when finished.

Screenshots:



Task 7: Integration and Demonstration

Prepare a coherent demonstration of approximately five minutes. Do not structure the program as a menu that simply runs one pattern example after another. The demonstration should tell a believable story in your chosen domain and naturally expose the important design decisions.

During the demonstration, a tutor may ask any team member to:

- identify where a GoF participant appears in the code;
- explain an ownership or lifetime decision;
- explain how traversal state is maintained and what happens after a runtime modification;
- trace an Activity Diagram action or decision to the implementation;
- explain a state transition or decorated behaviour;
- run or discuss GDB and Valgrind evidence;
- build/run the program in Docker; or
- discuss the team's Git history.

All three team members are responsible for understanding the complete submission.

Team contribution statement

Lilly

- set up the google doc
- set up Github repo
- implemented State and Decorator
- Activity diagrams

Lethabo

- Implemented composite and Iterator
- Task 5 - debugging and memory investigation
- UML

Francois

- Implemented main
- Dockerfile, readme
- Object and State diagrams

All team members discussed Task 1.
